



Amazon EC2 — Complete Notes

Elastic Compute Cloud · Resizable virtual servers on AWS

AWS EC2 Service Compute Pricing Pay as you go Scaling Elastic

A reader-friendly, all-in-one reference for Amazon EC2.

Table of Contents

1. What is EC2
2. Core Concepts
3. Instance Types & Families
4. AMI (Amazon Machine Image)
5. Purchasing / Pricing Options
6. Storage Options
7. Networking
8. Security Groups & NACLs
9. Key Pairs & IAM Roles
10. User Data & Instance Metadata
11. Placement Groups
12. Auto Scaling
13. Load Balancing
14. Instance Lifecycle
15. Monitoring & Status Checks
16. Pricing
17. Common CLI Commands
18. Key Limits & Numbers
19. Ways to Connect to EC2
20. Quick Mental Model

1. What is EC2

Amazon EC2 (**Elastic Compute Cloud**) provides **resizable virtual servers** ("instances") in the cloud. You launch a machine with a chosen OS, CPU, memory, storage, and network capacity in minutes, pay only while it runs, and scale up/down or in/out on demand.

💡 **Mental model:** EC2 is **Infrastructure as a Service (IaaS)** — AWS manages the physical host, hypervisor, and data center; **you** manage the OS, patches, runtime, and your application.

Why use it: full control of the OS, any software stack, predictable VM model, lift-and-shift of existing apps, and a huge range of instance sizes (from tiny burstable to GPU/HPC monsters).

Common uses: web/app servers, batch processing, dev/test, databases, game servers, ML training/inference, HPC, hosting almost anything you'd run on a physical server.

2. 🧩 Core Concepts

Concept	Description
Instance	A running virtual machine.
AMI	Amazon Machine Image — the template (OS + config + software) an instance boots from.
Instance Type	The hardware profile: vCPUs, memory, network, storage (e.g. <code>t3.micro</code> , <code>m6i.large</code>).
Region / AZ	Instances run in an Availability Zone within a Region.
Key Pair	Public/private SSH key for login (private key is yours; never re-downloadable).
Security Group	Stateful virtual firewall attached to the instance's network interface.
EBS Volume	Network-attached persistent block storage (virtual disk).
Elastic IP	A static public IPv4 you own and can remap between instances.
ENI	Elastic Network Interface — a virtual NIC (IP, MAC, security groups).
User Data	A script run at first boot to bootstrap the instance.
Tags	Key-value labels for organization, cost allocation, automation.

3. ⚙️ Instance Types & Families

Instance types are grouped into **families** optimized for different workloads. Naming: `m6i.large` → `m` family, `6` generation, `i` Intel (`a` =AMD, `g` =Graviton/ARM), `large` size.

Family	Optimized for	Examples	Use cases
General Purpose	Balanced CPU/mem	t3 , t4g , m6i , m7g	Web servers, small DBs, dev/test
Compute Optimized	High CPU	c6i , c7g	Batch, gaming, HPC, media encoding
Memory Optimized	High RAM	r6i , x2idn , u- (high mem)	In-memory DBs, caches, big data, SAP HANA
Storage Optimized	High local disk IOPS/throughput	i4i , d3 , im4gn	NoSQL, data warehouses, log processing
Accelerated Computing	GPUs / FPGAs / accelerators	p5 , g5 , inf2 , trn1	ML training/inference, graphics, HPC
HPC Optimized	Tightly-coupled HPC	hpc7g , hpc6id	Simulations, weather, genomics

T-family (burstable): t2 / t3 / t4g accumulate **CPU credits** when idle and spend them to burst. Great for spiky low-average workloads; watch out for credit exhaustion (or use *unlimited* mode). **Graviton (g suffix):** AWS ARM CPUs — better price/performance for many workloads.

4. 🖥️ AMI (Amazon Machine Image)

An AMI is the **template** an instance boots from: the root volume image (OS + software), launch permissions, and a block-device mapping.

- **Sources:** AWS-provided (Amazon Linux, Ubuntu, Windows...), AWS Marketplace, community AMIs, or **your own custom AMI**.
- **Custom AMIs** bake in your software/config → faster, repeatable launches (golden image).
- AMIs are **region-scoped** — copy an AMI to another region to use it there.
- Can be **private, shared with specific accounts, or public**.
- Backed by **EBS snapshots** (most common) or instance store.

🔧 Typical flow: launch instance → configure it → **Create Image** → launch many identical instances from that AMI (often via a Launch Template + Auto Scaling).

5. 💰 Purchasing / Pricing Options

Pick the model by commitment and tolerance for interruption.

Option	How it works	Savings	Best for
On-Demand	Pay per second/hour, no commitment	Baseline (most expensive)	Spiky / short-term / unpredictable workloads
Savings Plans	Commit to \$/hour for 1 or 3 yrs	Up to ~72%	Steady compute usage, flexible across types (Compute SP)
Reserved Instances (RI)	Commit to a specific type/region 1–3 yrs	Up to ~72%	Steady-state, known instance shape
Spot Instances	Bid on spare capacity, can be reclaimed with 2-min notice	Up to ~90%	Fault-tolerant, stateless, batch, CI, big data
Dedicated Hosts	A whole physical server for you	—	Licensing (BYOL), compliance, socket-based licenses
Dedicated Instances	Hardware isolated at the account level	—	Isolation without managing the host
Capacity Reservations	Reserve capacity in an AZ, no term commitment	—	Guaranteeing capacity (e.g. for failover/events)

💡 **Spot tips:** use Spot for interruptible work, handle the 2-minute interruption notice, and mix with On-Demand via **Auto Scaling mixed instances policy** or **EC2 Fleet / Spot Fleet**.

6. 🗄️ Storage Options

Storage	Type	Persists?	Notes
EBS (Elastic Block Store)	Network block volume	✅ Yes (independent of instance)	Most common root + data disk. Snapshot to S3. One AZ.
Instance Store	Local physical disk on host	❌ No (lost on stop/terminate)	Very fast/ephemeral; for caches, scratch, temp data.
EFS (Elastic File System)	Managed NFS (multi-AZ)	✅ Yes	Shared POSIX file system across many instances (Linux).
FSx	Managed file systems	✅ Yes	Windows File Server, Lustre (HPC), NetApp ONTAP, OpenZFS.
S3	Object storage	✅ Yes	Not a disk; accessed via API for backups, data, artifacts.

EBS volume types:

Type	Class	Use case
gp3 / gp2	General-purpose SSD	Default; boot volumes, most workloads (gp3 = baseline 3,000 IOPS, independent throughput)
io2 Block Express / io1	Provisioned IOPS SSD	High-performance DBs, low-latency, high IOPS
st1	Throughput-optimized HDD	Big data, log processing, streaming (sequential)
sc1	Cold HDD	Infrequent access, lowest cost

EBS volumes are **AZ-scoped**; **snapshots** (stored in S3, regional) let you back up, copy across AZs/regions, and create AMLs.

7. Networking

- **VPC** — your isolated virtual network. Instances launch into a **subnet** within an AZ.
- **Public subnet** → has a route to an **Internet Gateway (IGW)**. **Private subnet** → reaches the internet only via a **NAT Gateway**.
- **Private IP** — internal, stays with the instance for its life. **Public IP** — internet-facing, **changes on stop/start** (dynamic).
- **Elastic IP (EIP)** — a **static** public IPv4 you own; remap it between instances. (Charged when allocated but not in use.)
- **ENI** — virtual network card; an instance can have multiple ENIs / multiple IPs.
- **Enhanced Networking (ENA / SR-IOV)** and **placement groups** boost throughput / lower latency.
- **IPv6** supported; **Bandwidth** scales with instance size.

Connectivity to other networks: **VPC Peering**, **Transit Gateway**, **VPN**, **Direct Connect**, **PrivateLink** / **VPC endpoints**.

8. Security Groups & NACLs

A **Security Group** is a **virtual, stateful firewall** attached to an instance's network interface (ENI). It controls traffic with **allow rules** that match a **protocol + port + source/destination** (an IP/CIDR or another security group). Because it's **stateful**, if you allow a request in, the response is automatically allowed back out — you don't write a return rule. By default a security group **denies all inbound** and **allows all outbound**, and an instance can have several attached at once (their rules are combined). It's your primary, instance-level control; the NACL below is the coarser subnet-level backstop.

Two layers of network filtering — know the difference:

	Security Group	Network ACL (NACL)
Scope	Instance / ENI level	Subnet level
State	Stateful (return traffic auto-allowed)	Stateless (must allow both directions)
Rules	Allow only	Allow and Deny
Evaluation	All rules evaluated	Rules in numbered order , first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows all; custom denies all

✅ **Best practice:** reference **other security groups** as sources (not raw IPs) for tier-to-tier rules; open only the ports you need (e.g. 22/SSH, 443/HTTPS) and restrict source CIDRs.

9. 🔑 Key Pairs & IAM Roles

Key Pairs (SSH/RDP login):

- AWS stores the **public** key; you keep the **private** key (`.pem`). It is shown **only once at creation** — losing it means you can't SSH in (recover via other methods).
- Linux: SSH with the private key (`ssh -i key.pem ec2-user@ip`). Windows: decrypt the admin password with the private key for RDP.

IAM Roles / Instance Profiles:

- Attach an **IAM role** to an instance so apps on it get **temporary credentials** automatically (via the metadata service) — **never hard-code access keys** on an instance.
- The role defines what AWS APIs the instance may call (e.g. read an S3 bucket, write CloudWatch logs).

10. 📖 User Data & Instance Metadata

User Data

- A script (bash / PowerShell / cloud-init) passed at launch and run **once on first boot** to bootstrap (install packages, configure, pull code).
- Can be set to run on every boot with extra config.

Instance Metadata Service (IMDS)

- Reachable from inside the instance at `http://169.254.169.254/latest/meta-data/` — exposes instance id, AMI id, IP, IAM role credentials, user data, etc.
- **Use IMDSv2** (session/token-based) — more secure against SSRF than the legacy IMDSv1. Enforce it (`HttpTokens=required`).

```
# Example: fetch metadata using IMDSv2
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id
```

11. 📌 Placement Groups

Control how instances are physically placed to balance performance vs. resilience:

Strategy	Placement	Use for
Cluster	Packed close in one AZ	Low latency, high throughput (HPC, tight node-to-node)
Spread	Spread across distinct hardware/racks	Critical instances that must not share failure domains
Partition	Grouped into isolated partitions	Large distributed/replicated systems (HDFS, Kafka, Cassandra)

12. 📈 Auto Scaling

EC2 Auto Scaling Group (ASG) automatically adjusts the number of instances to match demand.

- **Launch Template** (preferred over launch config) defines *what* to launch (AMI, type, key, SGs, user data).
- **Min / Desired / Max** capacity bounds the group.
- **Scaling policies:** Target Tracking (e.g. keep CPU at 50%), Step scaling, Simple scaling, Scheduled scaling, Predictive scaling.
- **Health checks** (EC2 or ELB) replace unhealthy instances automatically → **self-healing**.
- Spread instances across **multiple AZs** for high availability.
- Integrates with **ELB** to register/deregister instances automatically.

💡 Auto Scaling = elasticity (right-size to load) **and** resilience (auto-replace failed nodes). It does **not** cost extra — you pay only for the instances.

13. ⚖️ Load Balancing

Elastic Load Balancing (ELB) distributes traffic across instances/targets in multiple AZs.

Type	Layer	Best for
ALB (Application LB)	L7 (HTTP/HTTPS)	Web apps, path/host routing, microservices, containers
NLB (Network LB)	L4 (TCP/UDP/TLS)	Ultra-high performance, static IP, low latency
GWLb (Gateway LB)	L3	Deploying virtual appliances (firewalls, IDS/IPS)
CLB (Classic, legacy)	L4/L7	Older accounts only — avoid for new work

Pair ELB + Auto Scaling across AZs for a highly available, self-healing tier. Use **health checks** so traffic only goes to healthy targets.

14. 🔄 Instance Lifecycle

States: pending → running → stopping → stopped → shutting-down → terminated (also rebooting).

Action	What happens	Data
Stop	Shuts down; EBS root persists; public IP released	EBS data kept; instance-store lost
Start	Boots again (maybe new host, new public IP)	EBS data intact
Reboot	Restart in place; stays on same host	Everything kept (incl. instance store)
Hibernate	RAM saved to EBS, resumes state on start	Memory preserved (size/AMI limits apply)
Terminate	Instance deleted	Root EBS deleted by default (unless "delete on termination" off); attached data volumes per setting

⚠️ **Stop ≠ Terminate.** Terminate is permanent. Enable **termination protection** for important instances. Instance-store data is **always lost** on stop/terminate.

15. 📊 Monitoring & Status Checks

- **CloudWatch metrics** (default, 5-min; **detailed** = 1-min, extra cost): CPU, network, disk I/O, status checks. *Note: memory & disk usage need the **CloudWatch agent**.*
- **Status checks:**
 - **System status check** — AWS infrastructure (host, network, power). Fix = stop/start (moves to new host).
 - **Instance status check** — the instance/OS itself (bad config, exhausted memory, kernel). Fix = reboot / fix OS.
- **CloudWatch Alarms** → trigger actions (notify, auto-recover, scale, stop/terminate).
- **EC2 Auto Recovery** for failed system checks; **CloudWatch Logs** + agent for app/OS logs.
- **CloudTrail** logs EC2 API calls for audit.

16. 💰 Pricing

You pay for:

1. **Instance compute** — per second (min 60s) or hour, by type & purchase option.
2. **EBS storage** — per GB-month provisioned, plus IOPS/throughput for io/gp3, plus snapshots.
3. **Data transfer OUT** — to internet (*IN is free; cross-AZ and inter-region transfer is charged*).
4. **Elastic IPs** — free while attached to a running instance; charged when allocated and idle.
5. **Extras** — detailed monitoring, ELB hours + LCUs, NAT Gateway, etc.

 **Cost tips:** right-size instances · use **Graviton** for price/performance · **Spot** for interruptible work · **Savings Plans/RIs** for steady usage · **stop** non-prod overnight · delete unused EBS volumes + old snapshots · release idle EIPs · use **Compute Optimizer** recommendations.

17. 📖 Common CLI Commands

```
# List instances (id, type, state)
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType,State.Name]' --output table

aws ec2 run-instances --image-id ami-123 --instance-type t3.micro \
  --key-name mykey --security-group-ids sg-123 --subnet-id subnet-123 # launch

aws ec2 stop-instances      --instance-ids i-123      # stop
aws ec2 start-instances     --instance-ids i-123      # start
aws ec2 reboot-instances    --instance-ids i-123      # reboot
aws ec2 terminate-instances --instance-ids i-123      # terminate

aws ec2 describe-instance-status --instance-ids i-123 # status checks
aws ec2 create-image --instance-id i-123 --name "my-ami" # bake an AMI
aws ec2 describe-security-groups      # list SGs
aws ec2 allocate-address               # get an Elastic IP

# Connect without SSH keys (needs SSM agent + IAM role)
aws ssm start-session --target i-123
```

18. 📌 Key Limits & Numbers

Item	Value
Billing granularity	Per-second (60s minimum) for Linux; per-hour for some
Spot interruption notice	2 minutes
Default running On-Demand limit	vCPU-based quota per family (adjustable)
Security groups per ENI	5 (default, up to 16 on request)
Rules per security group	60 inbound + 60 outbound (default)
EBS volume size	1 GiB – 64 TiB (per volume)
Max IOPS (io2 Block Express)	up to 256,000
Elastic IPs per region	5 (default, adjustable)
Metadata endpoint	<code>169.254.169.254</code> (link-local)
EBS snapshots	Stored in S3, incremental, regional

19. 🖱️ Ways to Connect to EC2

Method	How	Best for
SSH	<code>ssh -i key.pem ec2-user@<public-ip></code> (Linux)	Direct Linux access with a key pair + port 22 open
RDP	Decrypt admin password with private key, connect via RDP client (port 3389)	Windows instances
EC2 Instance Connect	Browser/CLI push of a temporary SSH key (Amazon Linux/Ubuntu)	Quick keyless-feeling SSH without managing keys
SSM Session Manager	<code>aws ssm start-session --target i-123</code>	No open inbound ports, no key pair — needs SSM agent + IAM role. Most secure; auditable.
Serial Console	Browser serial connection	Troubleshooting boot/network issues
Bastion / Jump host	SSH through a public bastion to private instances	Reaching instances in private subnets

🔒 **Best practice:** prefer **SSM Session Manager** — it removes the need for open SSH ports, bastions, and long-lived keys, and logs every session. Default OS users: Amazon Linux `ec2-user`, Ubuntu `ubuntu`, RHEL `ec2-user` / `root`, Debian `admin`.

Quick pick guide

Need	Use
Quick Linux shell, key in hand	SSH
Windows desktop	RDP
No keys / no open ports / audited	SSM Session Manager
One-off SSH without storing keys	EC2 Instance Connect
Instance won't boot / network broken	Serial Console
Private-subnet instance	Bastion host or SSM

20. Quick Mental Model

- **EC2 = rentable virtual servers.** You manage the OS and up; AWS manages the hardware.
- Choose an **AMI** (the image) + an **instance type** (the hardware) + a **subnet** (the network).
- **Security Groups** are your stateful firewall; open only what you need.
- Use **EBS** for persistent disks, **snapshots** for backups, **instance store** only for scratch.
- Pick a **pricing model**: On-Demand (flexible) · Spot (cheap + interruptible) · Savings Plans/RIs (steady).
- **Auto Scaling + ELB across AZs** = elastic, self-healing, highly available.
- Give instances an **IAM role**, never hard-coded keys; use **IMDSv2**.
- Prefer **SSM Session Manager** to connect — no open ports, no key sprawl.
- **Monitor** with CloudWatch + status checks; **right-size** relentlessly to control cost.

 Notes compiled as a quick reference for Amazon EC2.